



CODO: An Automated Compiler for Comprehensive Dataflow Optimization

Weichuang Zhang
School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
1064080006@sjtu.edu.cn

Yiquan Wang
School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
abcdfehg@sjtu.edu.cn

Xinzhou Zhang
School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
xz_zhang@sjtu.edu.cn

Chi Zhang
School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
zhang-chi@sjtu.edu.cn

Yu Feng
School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
y-feng@sjtu.edu.cn

Xiaofeng Hou
School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
hou-xf@cs.sjtu.edu.cn

Chao Li
School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
lichao@cs.sjtu.edu.cn

Jieru Zhao*

School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
zhao-jieru@sjtu.edu.cn

Minyi Guo
Guizhou Provincial Laboratory of Big Data
College of Computer Science
and Technology, Guizhou University
School of Computer Science, SJTU
guo-my@cs.sjtu.edu.cn

Abstract—FPGAs are well-suited for dataflow architectures that process data in a streaming or pipelined manner, thus satisfying the high computational and communication demands of emerging applications. However, manually implementing an efficient dataflow architecture for large-scale applications is still challenging, even for specialists who use high-level synthesis (HLS) to simplify FPGA programming.

To address this, we introduce CODO, an automated compiler that generates feasible and efficient dataflow accelerators on FPGAs. CODO features a systematic method for detecting and eliminating both coarse-grained and fine-grained dataflow violations. Building on this, CODO performs both on- and off-chip data movement optimizations to maximize transfer efficiency. To guarantee a higher design quality, CODO performs automatic scheduling to generate high-performance dataflow accelerators, ensuring a balanced performance-resource trade-off. Synthesis results show that CODO delivers $1.45\times$ to $4.52\times$ latency speedups on typical computation kernels and $3.7\times$ to $33.8\times$ speedups on DNN models compared to SOTA frameworks. In on-board evaluations, CODO achieves $7.3\times$ average speedup on CNN models and $2.07\times$ average speedup on the GPT-2 model over SOTA frameworks. The compiler is open-sourced at <https://github.com/sjtu-zhao-lab/codo-artifact>.

I. INTRODUCTION

Dataflow architectures are suitable for workloads that require massive data movement and operations due to their low latency [5], [14], [20]. Fundamentally, these architectures leverage task-level pipelining to allow distinct functions and

loops to overlap in their execution, rather than running sequentially. Furthermore, they exploit efficient on-chip communication between tasks [42], [44], significantly reducing the overhead of frequent external memory accesses [9], [30]. FPGAs, with their reconfigurable logic and customizable data paths, are well-suited for implementing dataflow accelerators that process data in a streaming or pipelined manner [6], [23], [26], [47]. Note that while the term *dataflow* is also used to describe dynamic scheduling-based *dataflow* circuits [21] or *dataflow* mapping strategies like input/output stationary [11], these concepts are conceptually orthogonal to the dataflow architecture discussed in this paper.

However, the high efficiency of dataflow accelerators comes at the expense of a complex design process using hardware description languages (HDLs). To simplify FPGA development, developers utilize high-level synthesis (HLS) to translate C/C++ code into HDL implementations automatically [12]. Nevertheless, a notable gap still persists between HLS programming and efficient dataflow implementations. Commercial HLS tools, such as AMD Vitis HLS [41], provide basic dataflow scheduling primitives, i.e., the *dataflow pragma* [4], to enable pipelined execution between loops or functions. However, this optimization works only if the coding styles satisfy stringent requirements [4], which many handcrafted algorithms fail to meet. This mismatch hinders effective optimization or even leads to synthesis failures. Consequently, developers must perform extensive code refactoring and optimization manually to convert algorithms into dataflow-feasible

* Corresponding author: Jieru Zhao.

formats and produce dataflow accelerators.

Prior Research. To reduce developing efforts, prior methods enhance programming efficiency using domain-specific languages (DSLs) [24], [40], [46], or directly parse C++ inputs or PyTorch models into intermediate representations (IRs) [19], [43], [49]. They primarily focus on kernel computation optimization, with limited consideration for dataflow optimization across multiple kernels or tasks [8], [10], [42], [44], enabling automatic generation of dataflow accelerators. However, these approaches fail to fully resolve potential issues (Fig. 2) in the dataflow, limiting their ability to further optimize code and exploit parallelism. As a result, the generated designs may suffer from suboptimal performance or even encounter deadlocks when deployed on FPGA boards.

Key Idea. The performance of dataflow accelerators is influenced by multiple factors. Fundamentally, the input code must satisfy strict constraints to enable correct streaming execution (*correctness*). On top of that, achieving high-throughput communication necessitates effective optimization of communication buffers and careful alignment of computation patterns between adjacent tasks to enhance data transfer efficiency (*communication*). Finally, balancing task latencies through techniques such as loop tiling, unrolling, and pipelining is essential for improving overall performance (*parallelism*).

The core problem is that these factors are deeply codependent, yet prior work typically handles them in a decoupled manner. This leads to conflicts where optimizing one aspect in isolation negatively impacts the others. For instance, aggressive code transformations to meet dataflow constraints may result in inefficient computation and memory access patterns that create a communication bottleneck. Conversely, communication or parallelism optimizations may violate dataflow constraints and produce invalid designs. Moreover, the growing scale and structural complexity of modern DNNs further exacerbate the problem, making it increasingly difficult to construct deep, high-throughput pipelines for large models.

To overcome these issues, we build a compiler that jointly co-optimizes correctness, communication, and parallelism. Through advanced code analysis, versatile optimization techniques, and automated scheduling, the compiler performs coordinated transformations that harmoniously benefit all three aspects rather than creating conflicts, automatically generating high-performance accelerators for large-scale models.

Challenges. Achieving the goal is challenging. Firstly, the input algorithm may violate dataflow constraints, resulting in *dataflow violations* that must be eliminated. At the coarse-grained level, existing HLS tools impose a strict single-producer-consumer constraint to enable dataflow optimization [4]. At the fine-grained level, producers and consumers must maintain consistent data access order and count to ensure correct and efficient streaming execution. Although recent works attempt to address these violations [8], [10], [42], [44], their methods are difficult to fully eliminate all violations in large-scale models. Consequently, unresolved violations result in *discontinuous dataflow regions*, breaking end-to-end

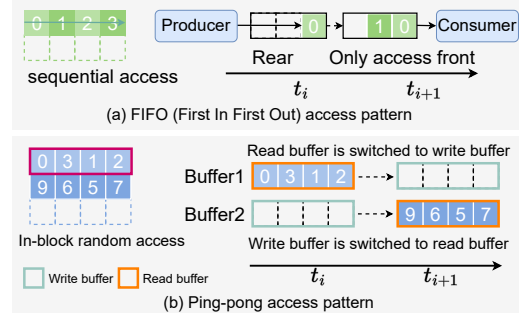


Fig. 1: Dataflow execution with FIFO and ping-pong buffer. Numbers in (a) and (b) represent the data access order.

streaming (Fig. 2, Issue 1).

Secondly, data is transferred through communication buffers, typically implemented as ping-pong buffers or FIFOs (First-In-First-Out), as shown in Fig. 1. To achieve high throughput, intermediate results must be produced and consumed just-in-time to prevent pipeline stalls. This requires careful selection of buffer types. Moreover, this idealized communication is often disrupted by other optimizations, such as violation elimination, which may unintentionally alter computation schedules and compromise communication efficiency. Without holistic coordination and advanced code analysis, such issues are easily overlooked [8], resulting in *delayed buffer writes and performance degradation* (Fig. 2, Issue 2).

Thirdly, improving dataflow performance requires balancing tasks through techniques such as loop tiling, unrolling, and pipelining. This becomes even more challenging in FIFO-based dataflow, as code optimizations affect data access patterns, requiring careful coordination between adjacent producers and consumers to avoid new dataflow violations. Consequently, existing auto-scheduling methods for ping-pong-based dataflow [44] are not directly applicable. While some tools use manual scheduling [24], [40] or nonlinear programming (NLP)-based methods [8], [33] for FIFO-based dataflow, they lack effective pruning methods to handle the exponentially increasing design space. As a result, these approaches *fail to scale to large-scale models*.

Our Solution. To tackle the above challenges, we propose CODO, an open-source compiler that performs comprehensive dataflow optimizations and automatically generates high-performance accelerators. CODO resolves dataflow violations, ensures communication efficiency, and explores resource-aware parallelism strategies to guarantee balanced task execution for large-scale models.

- We present an end-to-end compiler that automatically transforms an input algorithm into high-quality dataflow accelerators, along with the host code.
- We eliminate both coarse- and fine-grained dataflow violations and enable efficient data communication via on- and off-chip optimizations.
- We propose an automated scheduling method that rapidly determines suitable parallelism strategies with high resource efficiency to generate a high-performance design.

TABLE I: Comparison between representative compilers.

Feature	ScaleHLS [43]	POM [46]	Allo [10]	HIDA [44]	StreamHLS [8]	StreamTensor [42]	CODO
Compiler Front-end	PyTorch	DSL	PyTorch	DSL	PyTorch	PyTorch	PyTorch
Coarse-grained Violation Elimination	Limited	Manual	Manual	✓	✓	✓	✓
Fine-grained Violation Elimination	✗	✗	✗	✗	Limited	Limited	✓
Efficient Communication Buffer	✗	✗	✓	✓	✗	✗	✓
Resource-aware Node Balancing	✗	✗	✗	✗	✗	✗	✓
Automated Scheduling or DSE	✓	✓	✗	✓	✓	✓	✓
On-board Verification	✗	✗	✓	✗	✗	✓	✓
Open Source Project	✓	✓	✓	✓	✓	✗	✓

- We perform synthesis and on-board evaluations. CODO achieves $3.7\times$ to $33.8\times$ speedup on DNN models in synthesis and an average speedup of $7.3\times$ for DNNs and $2.07\times$ for GPT-2 on-board compared to SOTA compilers.

II. BACKGROUND AND MOTIVATION

A. Dataflow Architecture and its Violations

A dataflow architecture is a computing model where operations are triggered by the availability of input data, enabling efficient parallel execution.

FIFO vs. Ping-Pong Buffer. A critical aspect of dataflow architecture is the way of data processing and transmission between tasks. Two common patterns are shown in Fig. 1.

For **FIFO-based dataflow**, data elements are generated sequentially by the producer and streamed into a first-in, first-out buffer. The consumer then reads and processes the data one element at a time following their order of arrival. FIFO-based dataflow accelerators often achieve higher performance, since consumers can start execution immediately once the required data element is available. Resource utilization is also efficient, since only the necessary in-flight data needs to be stored.

In contrast, **ping-pong-based dataflow** groups data elements into blocks. Adjacent blocks are written alternately into two separate buffers, e.g., Buffer1 and Buffer2 in Fig. 1(b), leading to higher memory usage. While the producer writes to one buffer, the consumer reads from the other. Unlike FIFO, ping-pong-based dataflow inherently exhibits higher latency, as the consumer cannot begin execution until the entire data block is available. Nevertheless, data in each block can be accessed randomly, offering greater flexibility in data processing.

Commercial HLS tools enable dataflow processing on FPGAs using certain directives or pragmas, such as `#pragma HLS dataflow` in Vitis HLS. However, mapping computation graphs directly can lead to dataflow violations, which can be classified into *coarse-grained* and *fine-grained* categories.

Coarse-grained Dataflow Violation. Existing HLS tools enforce a strict single-producer, single-consumer constraint, requiring data to flow exclusively from one producer to one consumer throughout the design [4]. However, the topologies, data dependencies, and the inherent dataflow in real-world applications, such as DNN models, are complex, frequently violating this strict constraint and hindering the task-level parallelism. We term this violation as *coarse-grained dataflow violation*. While tools like Vitis HLS offer guidelines for handling such violations, developers must still manually refactor the code, making this process labor-intensive and error-prone, especially for complex applications.

Fine-grained Dataflow Violation. Existing frameworks [43], [44], [46] prefer ping-pong-based dataflow due to its flexibility in data access order, albeit with some performance loss. In contrast, FIFO-based dataflow can deliver higher performance but is more challenging to implement, as the strict sequential access constraint of FIFO demands fine-grained computation control and significant code refactoring. We term the corresponding violation as *fine-grained dataflow violation*. Specifically, mismatched **data access count** or **order** between producers and consumers can cause issues such as data loss, FIFO overflows or underflows, and even deadlocks. Worse still, these violations may not be detected during synthesis. Although HLS tools offer co-simulation to identify potential deadlocks, it can take days for large models, and even fail to detect issues reliably. Therefore, detecting and eliminating these violations at an early stage is essential yet challenging.

B. Related Work and Comparison

We compare representative FPGA compilers in Table I. Recent achievements in domain-specific languages (DSLs) for FPGA enhance productivity [13], [19], [22], [24], [40], [46]. Spatial [22] and Aetherling [13] provide specific coding styles or rewriting rules to generate high-performance HDL code in Chisel [7]. Dahlia [29] introduces an HLS language with a type system that explicitly tracks hardware resources and time ordering when resources are available. HeteroCL [24] and HeteroFlow [40] extend the TVM DSL, providing customization for computation, memory, and data movement. POM [46] delivers performance improvements through loop transformations and dependency analysis. Besides DSL, some approaches directly parse C++ or Python inputs into IRs or provide templates to generate efficient FPGA accelerators. PyLog [19] can process Python code and offer automatic pragma insertion. Sisyphus [34] proposes an optimization template that integrates code transformation, pragma insertion, and tile size selection into a single optimization problem. Prometheus [32] further enhances Sisyphus by providing unified optimization of computation and communication. ScaleHLS [43] parses PyTorch models into IRs based on MLIR for further optimization.

Despite programming efficiency and performance improvement, these frameworks primarily focus on optimizing kernel computation, with limited consideration for dataflow optimization of large-scale applications. For frameworks enabling automated scheduling or design space exploration (DSE) [43], [46], as shown in Table I, ScaleHLS partially mitigates coarse-grained dataflow violation by resolving multi-consumer violations for ping-pong-based dataflow, whereas POM relies on developers to manually manage data dependencies using

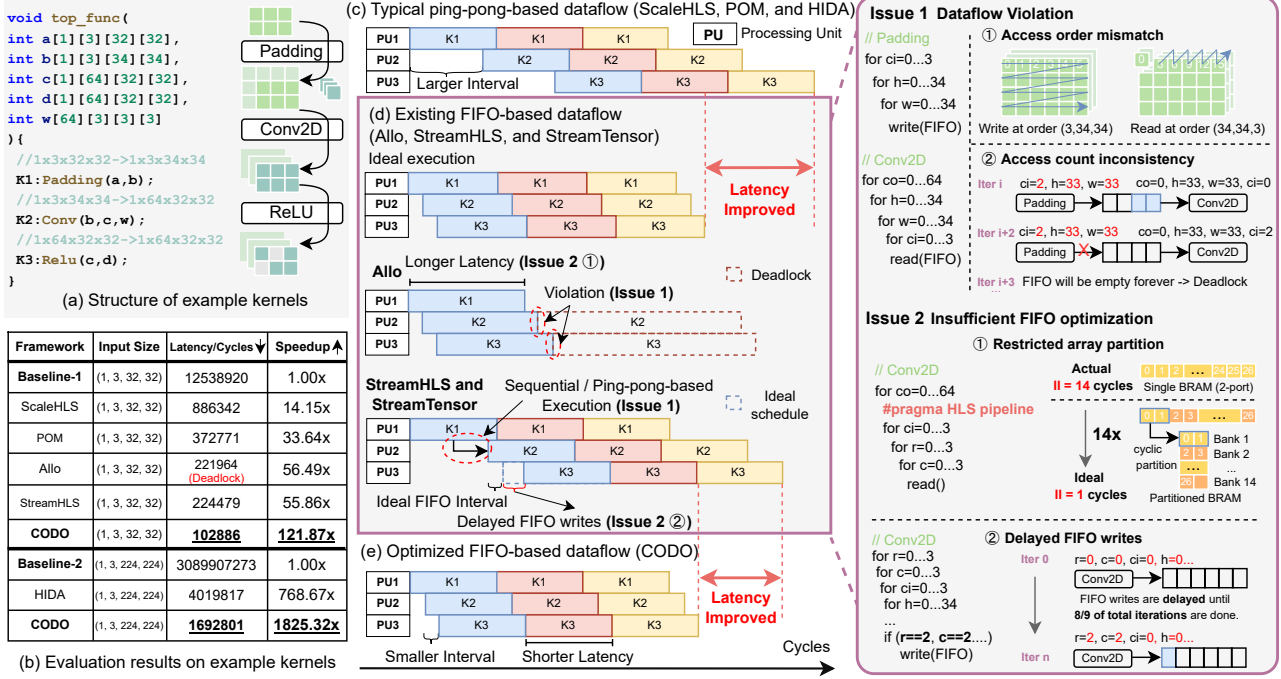


Fig. 2: Motivating example. (a) The code snippet consists of a top function and three sub-functions: Padding, Convolution, and ReLU. (c)(d)(e) present the schedules adopted by different frameworks, including ScaleHLS [43], POM [46], HIDA [44], Allo [10], StreamHLS [8], StreamTensor [42], and CODO. K1, K2, K3 correspond to example kernels. Different colors distinguish different iterations. (b) shows the performance of code without optimization (baseline) as well as the optimized code generated by six open-source compilers. The target device is AMD Alveo U280 FPGA [1]. The performances are *HLS synthesis results*.

DSL to prevent violations. However, both methods apply homogeneous optimization strategies across different tasks, e.g., layers of DNN models, resulting in suboptimal designs.

To boost performance, compilers for dataflow optimization are proposed [8], [10], [15], [42], [44]. TAPA [15] provides specialized APIs to express dataflow. However, it requires manual scheduling and lacks in-depth exploration of parallelism. Allo [10] features a composable DSL for efficient spatial accelerator generation with FIFOs. However, it overlooks dataflow violations in the design, potentially incurring pipeline stalls or even deadlocks. HIDA [44] targets ping-pong-based dataflow accelerators, eliminates coarse-grained dataflow violations, and applies dataflow parallelization automatically, but its processing in data blocks still results in suboptimal performance. StreamHLS [8] partially resolves dataflow violations through reordering loops and adding control statements for a single task. However, lacking advanced code analysis and scalable optimization methods, they fail to fully address violations for large-scale models featuring complex code patterns and deep layer hierarchies. StreamTensor [42] introduces an iterative tensor-based type system for dataflow optimization, but it falls back to a conservative ping-pong buffering strategy when encountering fine-grained dataflow violations, limiting its ability to exploit high-performance FIFO-based designs.

In contrast, CODO enables efficient streaming execution by eliminating dataflow violations at both coarse-grained and fine-

grained levels through advanced code analysis and transformation, applying extensive memory optimization for efficient data communication, and providing automated resource-aware scheduling to reduce overall latency with minimal resource overhead. CODO automatically generates efficient dataflow accelerators with host code for control, which can be directly deployed on an FPGA board.

C. Motivating Example

To further illustrate differences, we use a motivating example with three tasks (kernels), as shown in Figure 2(a). The dataflow patterns in existing works can be categorized into ping-pong-based and FIFO-based approaches.

Typical ping-pong-based Approaches. As shown in Fig. 2(c), POM [46], ScaleHLS [43], and HIDA [44] typically employ a ping-pong-based dataflow schedule. This approach requires waiting for an entire data block to be produced before subsequent kernels can start execution, resulting in long intervals and limited opportunities for overlapping operations. In contrast, as illustrated in Fig. 2(d), an ideal FIFO-based execution allows consumers to begin processing immediately once the required data element is available, resulting in a smaller interval between kernels and shorter overall latency.

Existing FIFO-based Approaches. While FIFO is an ideal pattern for large-scale dataflow accelerators, it presents significant challenges for designers. Figure 2(d) shows schedules adopted by Allo [10], StreamHLS [8], and StreamTensor [42].

Allo [10] presents meaningful attempts to enable FIFO-based dataflow, but two critical issues hinder their on-board execution. First, for the Padding and Conv2D tasks, two dataflow violations arise (Issue 1). The first violation occurs due to a mismatch of the data access order between adjacent tasks: Padding, as the producer, writes to the FIFO in the loop order of (3,34,34), while Conv2D, the consumer, reads from the FIFO in the loop order of (34,34,3). This discrepancy violates the consistent sequential accessing constraint of FIFOs. The second violation stems from an inconsistency in the data access count between tasks. As shown in the right part of Fig. 2, after Padding writes its last data at iteration i , no additional data is written to the FIFO, yet the consumer (Conv2D) is still waiting. This inconsistency leads to a deadlock after iteration $i+2$. Second, existing works, including [10], do not fully explore memory optimization with array partitioning (Issue 2 ①). For example, in Conv2D shown in the right part of Fig. 2, Allo [10] employs a pipeline pragma on the outermost loop, causing all inner loops to be unrolled. However, without proper array partitioning, all $3 \times 3 \times 3$ elements are sequentially read from a dual-port BRAM, resulting in a 14-cycle delay.

StreamTensor [42] introduces a type system for dataflow optimization. When types of a producer and consumer match, FIFO optimization can be performed. However, in cases of mismatch as illustrated in Issue 1, it falls back to suboptimal ping-pong executions rather than resolving underlying violations. StreamHLS [8] provides partial solutions for Issue 1 by reordering loops and inserting control logic to enable FIFO-based dataflow. However, this method does not ensure that FIFO writes start as early as possible, and some cases may require further loop rewriting to achieve effective dataflow. Consequently, its limited code pattern analysis and transformation lead to suboptimal designs. First, it fails to identify and enable FIFO-based dataflow between Padding and Conv2D, reverting to ping-pong or sequential execution. Second, the data transfer process is inefficient due to a misalignment between the control logic and the loop execution order. As illustrated in Fig. 2 (Issue 2, ②), the control condition ($r=2$, $c=2$, ...) defers all FIFO write operations until a late stage within the iteration sequence. More precisely, FIFO writes are postponed until nearly 8/9 of the total iterations have completed. This delay introduces substantial pipeline bubbles, leaving the consumer task idle for most of the time. Quantitative comparison is shown in Fig. 2(b).

CODO Approach. CODO resolves all the aforementioned issues. It eliminates inconsistencies in data access between adjacent kernels and tasks through advanced code analysis and transformation, and employs communication buffer optimization to enable stable and efficient streaming processing on actual FPGA boards. In addition, CODO performs automated, resource-aware dataflow scheduling, achieving high parallelism without overusing limited hardware resources, while carefully coordinating neighboring producers and consumers to prevent new dataflow violations. In this way, CODO automatically generates highly efficient and deployable dataflow accelerators, demonstrating superior effectiveness and practi-

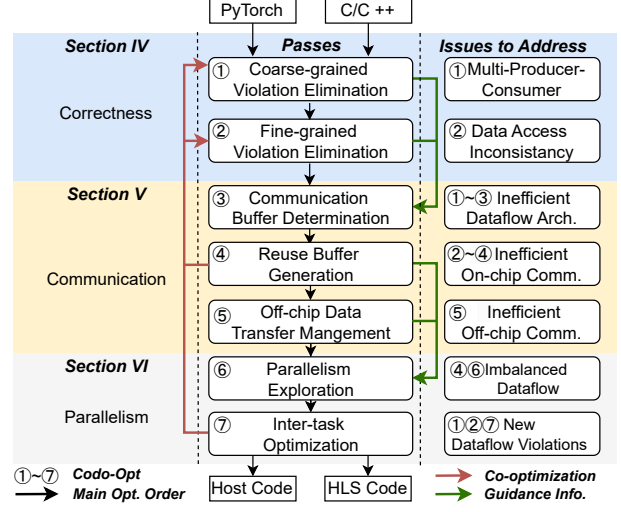


Fig. 3: Framework overview of CODO.

cal usability compared to prior methods.

III. FRAMEWORK OVERVIEW

CODO is built on the MLIR [25] compilation framework. Figure 3 shows the compilation flow. The framework takes compute kernels implemented in C++ or PyTorch models as input, which are translated into MLIR dialects via Polygeist [28] and Torch-MLIR [3], respectively. CODO offers `codo-opt`, which applies the full optimization flow in a single command, allowing users to optionally adjust input parameters like maximum parallelism and tiling factors.

CODO contains a holistic compilation flow that follows a main optimization order while being deeply integrated through co-optimization. The flow begins with two dataflow correction passes. The coarse-grained violation elimination resolves single-producer-consumer violations between tasks, where each task is represented as a *node* in the dataflow graph. Subsequently, the fine-grained violation elimination fixes inconsistencies in data access order and count, enabling efficient FIFO-based communication. This pass exemplifies our co-optimization principle: beyond ensuring correctness, it proactively restructures code for communication efficiency and provides guidance for later communication passes. Based on this, CODO performs communication buffer determination, selecting either FIFO or ping-pong implementations and prioritizing FIFO whenever feasible for higher performance. To further improve communication efficiency, CODO generates efficient reuse buffers and reinvokes the correctness passes to avoid new violations. This process also exposes loop-level parallelism, providing key information for subsequent parallelism exploration. Afterward, CODO manages off-chip transfers to improve HBM bandwidth utilization. Finally, CODO's auto-scheduling engine determines tiling factors, unroll factors, pipelining, and array partitioning. These parallelism decisions are not made in isolation, as they can affect both correctness and communication efficiency. Therefore, a final inter-task optimization pass co-optimizes these choices across the

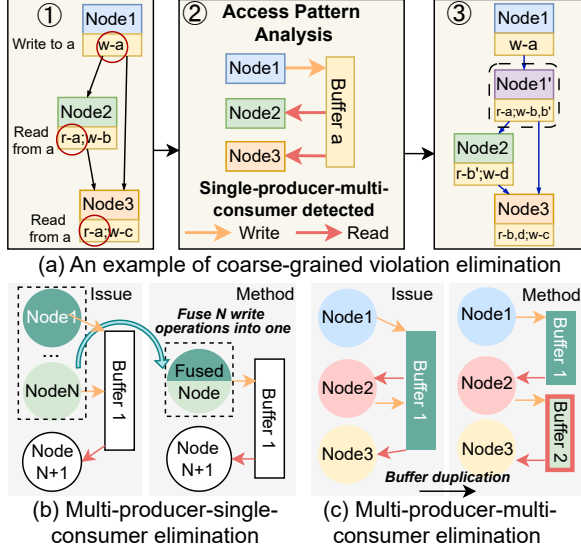


Fig. 4: Coarse-grained dataflow violation elimination: (a) an example, where $r\text{-}x/w\text{-}x$ represent read/write to buffer x , and $\text{Node1}'$ represents the node inserted to eliminate the violation; (b)(c) illustrate our elimination techniques for the rest of the dataflow violation categories.

entire graph, eliminating any newly introduced violations and ensuring a high-performance design.

IV. DATAFLOW VIOLATION ELIMINATION

Commercial HLS tools [2], [41] exhibit limitations in effectively addressing dataflow violations. These tools only report coarse-grained dataflow violations through synthesis analysis and cannot automatically transform the code to resolve violations. To address these issues, we systematically eliminate both coarse-grained and fine-grained dataflow violations.

A. Coarse-grained Violation Elimination

Violation Issues. The input C/C++ code or PyTorch models are first translated into a dataflow graph, where nodes represent computational tasks such as loops or functions, as shown in Fig. 4. Existing commercial HLS tools enforce a single-producer-single-consumer pattern for dataflow execution, as discussed in Section II-A. Therefore, effective techniques are necessary to eliminate violations that deviate from this constraint. Figure 4 illustrates different types of coarse-grained dataflow violations. For example, in Fig. 4(a), Node1 writes results to buffer a , while both Node2 and Node3 read from the same buffer, forming a single-producer-multi-consumer pattern. Similarly, Fig. 4(b) and Fig. 4(c) depict multi-producer-single-consumer and multi-producer-multi-consumer patterns, respectively. Although previous works [8], [43], [44] partially address violations (a) or (c), they often fail to eliminate all violations, leading to sequential execution between nodes.

Pattern-aware Code Transformation. To fully address these issues, we propose pattern-aware code transformation, as described in Algorithm 1. The algorithm traverses the input code and detects data access patterns that may lead

Algorithm 1 Pattern-aware Violation Elimination

Input: Initial input code M with nodes and buffers.

Output: Transformed code M' without violations.

```

1: for all  $buf \in M$ :
2:   Collect all nodes  $N$  that access  $buf$ .
3:    $V \leftarrow \text{analyze\_access\_pattern}(N)$ 
4:   if  $V$  contains violations:
5:     Detect the data access pattern  $P$ .
6:      $N' \leftarrow \text{apply\_transformation}(N, P)$ 
7:      $M' \leftarrow \text{update\_affected\_nodes}(M, N')$ 
8: return  $M'$ .

```

to violations (L3-4), which arise when *multiple nodes access the same buffer*. In general, all the access patterns that cause coarse-grained dataflow violations can be classified into three categories, as shown in Fig. 4. Once a violation is identified, CODO detects its access pattern and applies corresponding transformations to refactor the code (L5-6). For instance, Fig. 4(a)① illustrates a typical bypass pattern, commonly seen in models with residual structures such as ResNet-18 [16] and GPT-2 [35]. CODO begins by traversing all buffers and collecting all nodes that access them. Taking buffer a as an example, its relevant nodes are Node1-3 . CODO analyzes and records the access behavior of each node for buffer a (Fig. 4(a)②), which is then identified as the single-producer-multiple-consumer pattern. To resolve this violation, an intermediate node ($\text{Node1}'$) is inserted, reading from buffer a and writing to duplicated buffers b and b' (Fig. 4(a)③).

CODO applies different code transformations to address all three coarse-grained violation patterns in Fig. 4. The *multi-producer-single-consumer* pattern in Fig. 4(b), often found in initialization and padding operation pairs, is resolved through *node fusion*. CODO fuses loops that write to the same buffer when they share the same outer iteration domain and have no loop-carried dependencies. If inner loop structures differ, additional control logic is inserted to handle the mismatch. To maintain correctness, intermediate results from earlier writes are temporarily stored and finally merged into the last write operation. For the multi-producer-multi-consumer issue in Fig. 4(c), we create a new *buffer2* by duplicating *buffer1*, ensuring that each buffer is read from and written to once.

B. Fine-grained Violation Elimination

After the coarse-grained violation elimination, HLS tools can by default allocate ping-pong buffers between nodes to enable coarse-grained dataflow execution with data blocks. However, the performance may not be maximized at nodes whose input and output data can be transferred through FIFOs in sequential order and processed at a finer granularity. This is because FIFO-based dataflow often offers superior performance due to its streaming computation pattern and less resource overhead. However, it imposes strict requirements on code patterns, requiring fine-grained violation elimination.

Violation Issues. In the example of Fig. 4(a)③, FIFOs can be inserted at all the connections between nodes or loops

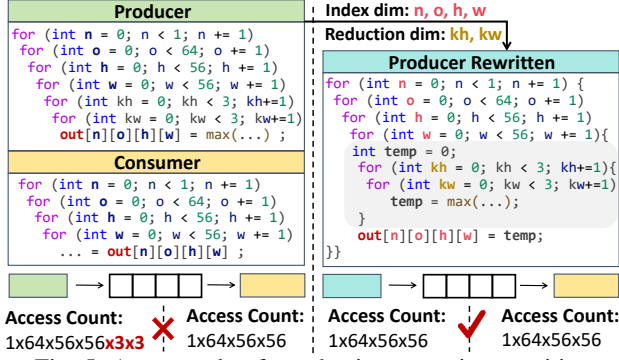


Fig. 5: An example of a reduction operation rewriting.

within nodes, only if the sequential data access constraint is satisfied and the data access orders/counts are consistent between adjacent nodes or loops. Unfortunately, real-world applications exhibit numerous fine-grained read-write inconsistencies. As discussed in Section II-C, violations such as *access count mismatch* and *access order inconsistency* can result in deadlock or computational errors in the final design. More critically, existing HLS tools cannot detect these violations during synthesis. While a subset of issues may be identified through cosimulation, the process is time-consuming, often taking days or even weeks, thereby significantly increasing the debugging burden for developers.

Systematic Read-Write Coordination. To address fine-grained violations and further refine the design for higher efficiency, we introduce a systematic read-write coordination method, including 1) *reduction operation rewriting*, which resolves the *data access count mismatch* issue while *guaranteeing early FIFO writes*, and 2) *permutation map generation*, which adjusts the access pattern of adjacent loops and ensures their consistent data access order, resolving the *data access order inconsistency* issue.

1) *Reduction Operation Rewriting.* Most data access count mismatches stem from reduction operations, such as fully connected layers, max pooling, and normalization. These non-bottleneck operations introduce loop dimensions that do not directly correspond to array indices, resulting in redundant FIFO accesses during reduction iterations. To address this issue, we propose a *reduction rewriting strategy* that identifies reduction regions and utilizes temporary arrays to aggregate intermediate results, thereby minimizing unnecessary FIFO transactions and ensuring correct and efficient dataflow execution.

Figure 5 illustrates a FIFO access mismatch and our approach. In this example, the producer is a max pooling operation that writes to buffer `out`, while the consumer is an initialization operation that reads from the same buffer. A discrepancy between the number of writes and reads results in a data access count mismatch, which leads to a FIFO deadlock. To detect such cases, CODO analyzes the loop structures of both the producer and the consumer that access the same array. It determines the total number of writes and reads by identifying the loop level at which the target array is accessed and computing the product of the iteration counts of

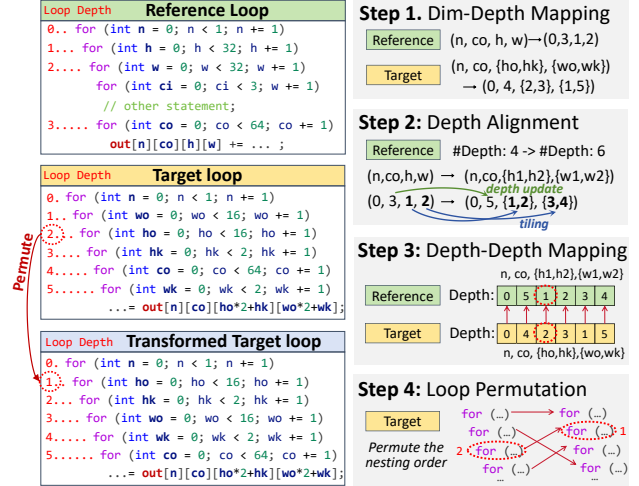


Fig. 6: Illustration of permutation map generation.

the surrounding loops. When a mismatch is detected, CODO classifies loop dimensions that correspond to FIFO array indices as *index dimensions*, while the remaining ones are identified as *reduction dimensions* and moved to the innermost loops, as shown in the shaded region of Fig. 5. The write to `out` is then moved out of the reduction region, and a temporary buffer is introduced to accumulate intermediate results. This transformation ensures that the producer's access count matches that of the consumer. Moreover, the rewriting ensures that intermediate results are being calculated and transferred just-in-time, greatly improving data transfer efficiency.

2) *Permutation Map Generation.* Inconsistent data access orders, which are common in real-world applications, lead to dataflow violations for streaming processing with FIFOs, as illustrated in Issue 1 of Fig. 2. To address this issue, we propose *permutation map generation*. Specifically, CODO identifies the bottleneck loop (e.g., convolution or Q*K in attention) as the *reference loop* by analyzing the trip counts and computational intensity of each nested loop. It then analyzes data access patterns of the *reference loop*, including the data access order of input and output arrays. This information serves as the basis for adjusting the data access patterns of its producer and consumer loops, termed *target loops*. CODO then employs a mapping-based strategy to efficiently align data access patterns between reference and target loops.

Figure 6 illustrates this process. In Step 1, CODO establishes a mapping from connection array dimensions to their corresponding loop depths for both reference and target loops. For example, in the reference loop, the dimension set $\{n, co, h, w\}$ of `out` corresponds to the loop depth set $\{0, 3, 1, 2\}$. In Step 2, we apply loop tiling with a tiling size of 1 to the reference loop to align the depths of the reference loop and the target loop, splitting `h` and `w` into two loops, respectively. In Step 3, we construct a mapping between the loop depth sets of the reference and target loops. For instance, a mapping from 2 to 1 indicates that the loop at depth 2 in the target loop should be swapped to depth 1. Finally, in Step 4,

we transform the target loop by permuting the nesting order based on the depth-depth map from Step 3.

V. EFFICIENT DATA COMMUNICATION

After eliminating dataflow violations, the input algorithm is transformed into a dataflow-feasible form. Based on it, optimizing both on-chip and off-chip data communication is critical for overall efficiency. Therefore, we propose two on-chip optimizations: 1) *communication buffer determination*, which prioritizes FIFOs for tasks without dataflow violations; 2) *violation-free reuse buffer generation*, which enhances data transfer efficiency while ensuring violation-free designs; and an off-chip optimization: 3) *off-chip data transfer management*, which improves HBM bandwidth utilization.

A. On-chip Communication Buffer Determination

We adopt a FIFO-first strategy to optimize on-chip communication buffers. For tasks free of violations, we prioritize FIFO implementations to maximize performance. If fine-grained violations between loops cannot be eliminated, we turn to ping-pong buffer implementations. Note that ping-pong buffers are more resource-intensive, as they require at least twice the buffer size of the transmitted data block, posing a risk of resource overflow in large-scale applications.

B. Reuse Buffer Generation

Leveraging reuse buffers is effective to enhance data reuse and memory bandwidth, but integrating them into the design requires careful loop refactoring. Existing HLS frameworks often rely on user-defined DSLs, requiring developers to manually specify buffer sizes, communication buffer types, and address mappings for reused data [10], [24], [40]. This approach demands developer expertise and often leads to sub-optimal or infeasible designs, as discussed in Section II-C. To this end, we propose a *violation-free reuse buffer generation* method that automatically exploits data reuse opportunities.

Violation-free Reuse Buffer Generation. To automatically generate efficient reuse buffers, CODO first analyzes the nested loop structure and the operations within each loop to identify computation-intensive kernels such as convolution and matrix multiplication. This is achieved by detecting common computation patterns, such as multiply-accumulate operations. It then extracts the input/output access patterns of the target array and analyzes the mapping between loop variables and array indices. Loop dimensions that appear in the array indices are identified as FIFO dimensions, while the remaining loop dimensions are treated as reduction dimensions. This information is guidance for subsequent reuse buffer generation. Taking the convolution example in Fig. 7, each output pixel depends on a small local region of the input feature map, and neighboring outputs reuse many input elements. These reuse opportunities align with the reduction dimensions. CODO automatically analyzes FIFO indices, identifies reduction dimensions that are independent of the FIFO, and constructs reuse buffers accordingly. Specifically, CODO constructs line

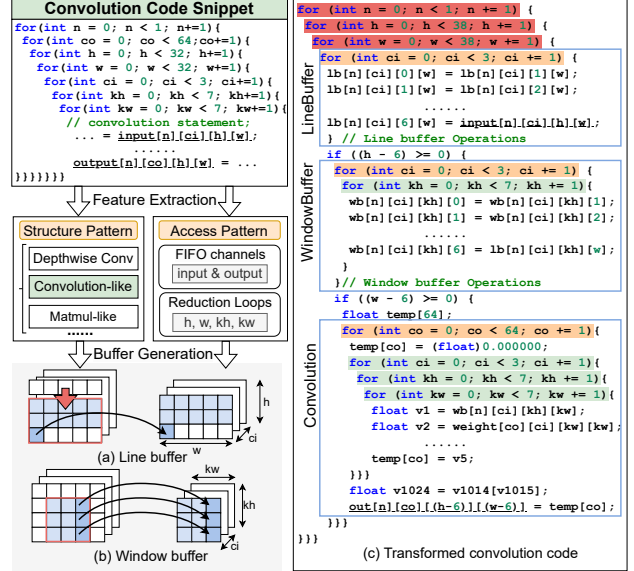


Fig. 7: Example code for efficient reuse buffer generation.

and window buffers based on the iteration domain of reduction loops (Fig. 7(a),(b)). The line buffer, denoted as $lb[n][ci][kh][w]$, stores multiple rows of the input feature map. Its depth is equal to the kernel height (kh), retaining $kh-1$ rows to preserve history for subsequent computations. Each new input element ($input[n][ci][h][w]$) is written into the most recent position. The window buffer, denoted as $wb[n][ci][kh][kw]$, maintains the full $kh \times kw$ window of the convolution kernel. For each new column w of the input, it updates by shifting existing contents horizontally and loading the new column from the line buffer. To prevent dataflow violations, CODO refactors loops while analyzing access patterns of FIFOs, ensuring that all loop dimensions are properly utilized. Specifically, loops involving FIFO accesses must neither include irrelevant dimensions nor omit necessary ones. For example, in Fig. 7 (c), the *input* and *output* arrays are optimized as FIFOs, and the nested loops enclosing them precisely align with the array indices, ensuring consistent data accesses. Note that this method is also applicable when the target array is implemented using ping-pong buffers.

Guidance for Parallelism Exploration. After reuse buffer generation, the rewritten code is ready for further optimizations through loop tiling, pipelining, unrolling, and array partitioning. However, as shown in Fig. 7, the generated loop is highly complex, making optimization challenging. By analyzing the internal computation behavior, we identify distinct parallelism opportunities. First, parallelizing the outermost red loop is unsafe, as it would unroll all three internal regions, introducing complex data dependencies and control issues. Second, the middle orange loops are associated with FIFO indices, and optimizing them could alter FIFO access patterns, potentially causing new violations. Finally, the innermost green loops are independent of FIFO behavior, making them safe for parallelization without introducing new violations.

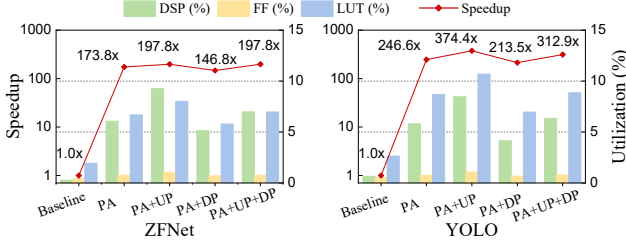


Fig. 8: Speed and resource util. of parallelism exploration.

Based on the above analysis, a loop is legal to parallelize if it has no loop-carried dependencies. If the target loop variable appears in FIFO indices, parallelization remains feasible, but additional measures are needed to preserve the consistency of the data access pattern between producer and consumer. This analysis is crucial for the subsequent parallelism exploration, as it enables the effective pruning of the vast design space associated with large-scale models.

C. Off-chip Data Transfer Management

To improve off-chip bandwidth utilization, CODO automatically constructs efficient burst transfers between HBM and on-chip memory. It distributes parameters such as model weights across different HBM channels, enabling parallel access to independent memory regions. CODO provides a `codo-transmit` command, which automatically generates the host code and burst-access operations for kernels and users can specify the number of HBM channels allocated.

VI. AUTOMATED DATAFLOW SCHEDULING

After applying previous passes, dataflow violations are eliminated and buffers are inserted at suitable positions. Then CODO performs auto-scheduling to exploit parallelism without exceeding resource budgets and coordinate adjacent tasks without introducing new dataflow violations.

Challenges. Parallelism exploration in dataflow accelerators presents significant challenges: 1) improper parallelism strategies can disrupt latency balance between tasks, leading to degraded dataflow performance; 2) existing methods prioritize performance gains while neglecting the resource-performance tradeoff, which can result in excessive resource consumption; 3) certain parallelization strategies may alter FIFO access patterns, introducing new dataflow violations.

Parallelism Exploration. To address these challenges, we propose resource-aware bottleneck-centric design space exploration (DSE) to find optimal parallelism strategies, including loop tiling, pipelining, unrolling, and array partitioning configurations. Fig. 8 illustrates the speedup and resource utilization at each stage of the parallelism exploration process.

1) *Stage One: Initial Parallelism Allocation (PA).* The DSE begins by constructing a high-quality initial design with initial parallelism degrees. CODO employs a profiling-based performance model [48] [43]. Latencies and resource consumption of basic operations, such as adders, are profiled, serving as the performance model parameters. Then the latency of each

loop can be estimated based on their loop trip counts and parallelism strategies. After estimating the latency of each loop, CODO allocates parallelism degrees in proportion to their latencies, setting the smallest degree to 1. It then gradually scales up the parallelism of all loops while preserving their proportional ratios until reaching the user-specified upper bound or hardware resource limits. This process helps form a roughly balanced dataflow structure.

Unlike methods that parallelize loop dimensions randomly, CODO leverages insights from the earlier communication optimization pass. It prioritizes tiling loop dimensions that are independent of FIFO accesses, ensuring correct and efficient communication, and automatically applies HLS pragmas such as pipelining, unrolling, and array partitioning to generate the initial design. As shown in Fig. 8, this stage brings significant benefits, delivering latency speedups of 173.8 $\times$ on ZFNet [45] and 246.6 $\times$ on YOLO [36].

2) *Stage Two: Upscaling (UP).* We then traverse all *bottleneck loops* and re-estimate their latencies. If one loop’s latency remains at least n times larger than the lowest loop latency, we increase its parallelism degree to $\max\{[n] \times \text{initial degree}, \text{max degree}\}$ and search corresponding parallelism strategies. This process is iterative and terminates when the parallelism degree of all loops stabilizes or the iteration limit is reached. Note that this step aims to maximize the parallelism degree for each bottleneck loop, which may increase resource usage. As shown in Fig. 8, after *PA+UP*, the performance of both models is improved at the cost of extra resource consumption.

3) *Stage Three: Downscaling (DP).* Since the overall performance of a FIFO-based dataflow depends on the loop with the longest latency, we introduce a downscaling step to adjust over-optimized loops. If a loop is n times faster than the longest loop, it indicates that this loop has been over-optimized. In this case, we decrease the parallelism degree of the loop by a factor of n , reducing resource usage while maintaining a balanced dataflow. This step may slightly reduce overall performance, but users can configure CODO to enable or disable it based on their design goals. As shown in Fig. 8, after applying *PA+UP+DP*, ZFNet achieves the same speedup as the *PA+UP* design but with significantly lower resource consumption. Users can also bypass the *UP* stage and directly perform *DP* to pursue the most resource-efficient design, particularly in resource-constrained scenarios.

The parameter n acts as a balancing threshold in the parallelism exploration process. In practice, increasing loop parallelism through transformations such as loop unrolling typically has a minimum granularity of 2. Therefore, we empirically set $n = 2.0$ to avoid skipping potentially optimal design points that might be missed if larger values (e.g., 3, 4, or 8) were used. We also observe that small variations around this value do not lead to noticeable performance differences. Developers can specify a customized value for n in `codo-opt` to adjust the exploration granularity based on their own design constraints.

Inter-task Optimization. After parallelism exploration, the

bottleneck loops are effectively optimized. However, if loop tiling is applied to dimensions used by FIFO, the loops connected to the other end of FIFO need to adopt the same parallelism strategy to maintain consistent data access behavior. CODO implements this by propagating the chosen loop tiling, unrolling, and array partitioning strategies of the bottleneck loop to its connected producer/consumer loops. These changes often reshape the loop structure, so we re-invoke our correctness passes to detect and resolve any newly introduced dataflow violations.

However, conflicts can arise during this stage. For example, consider a dataflow consisting of loops A, B, C, and D connected by FIFOs. If loops B and D adopt conflicting parallelism strategies, loop C may encounter an unresolvable violation. In such cases, CODO downgrades the buffer between loops C and D to a ping-pong buffer implementation, preserving the FIFO-based execution from loop A to loop C.

VII. IMPLEMENTATION

A. Frontend

CODO is seamlessly integrated into the MLIR infrastructure and supports two primary input pathways: (1) Polygeist [28] for translating C/C++ kernels directly into the affine dialect, and (2) Torch-MLIR [3] for importing PyTorch models. In the Torch-MLIR flow, models are first lowered to the linalg dialect, where built-in MLIR passes such as element-wise operator fusion and tensor bufferization are applied. The IR is then further lowered to the affine dialect, which serves as the primary representation for subsequent CODO optimizations. CODO targets affine programs with constant loop bounds, which covers a wide variety of kernels and layers in DNNs, linear algebra, and image processing, such as convolution, attention, activation layers (ReLU, GeLU), matrix multiplication, dot product, etc.

B. Optimization Passes

All optimizations are implemented as modular MLIR passes, enabling high extensibility, as illustrated in Fig. 3. After each transformation, MLIR’s built-in verification checks IR validity, including dominance relations, SSA consistency, and type correctness. In addition, the `canonicalize` and `cse` (common sub-expression elimination) passes remove dead code and redundant computations [27]. Therefore, the MLIR infrastructure inherently guarantees IR validity throughout the compilation process. Loop transformation passes, including violation elimination and reuse buffer generation, operate primarily on the affine dialect. To optimize data communication, CODO introduces dedicated data types and operations to explicitly model FIFO and ping-pong buffers. Hardware-specific semantics, such as dataflow pragmas and array partitioning directives, are represented through an extended HLS dialect based on the version originally developed by HIDA [44].

C. Translation

After optimizations, the transformed IR is lowered to the host code and the HLS C++ kernel code. To ensure functional correctness, we adopt the verification pipeline from

TABLE II: Evaluation on typical kernel-level applications.

Benchmark	DSP	Latency Speedup			
		CODO	StreamHLS	Allo	HIDA
Atax	602	853.3 ×	640.1×	293.2×	11.1×
Gesummv	562	369.1×	382.4 ×	329.1×	79.8×
Gemm	826	500.8×	600.4 ×	177.4×	239.2×
Mvt	600	488.1 ×	368.7×	249.5×	79.7×
3mm	830	379.0×	442.3 ×	321.0×	180.5×
Residual MLP	648	449.2 ×	449.2×	—	—
Autoencoder	696	329.4 ×	329.3×	7.7×	141.4×
Residual Block	488	225.8 ×	99.1×	—	127.7×
DWConv. Block	495	14.4 ×	7.7×	—	—
3-Layer Conv. Block	613	209.0 ×	37.2×	122.1×	—
Feed Forward	604	513.2 ×	256.6×	3.8×	—
Multi-Head Attention	848	256.5 ×	168.5×	4.0×	—
DSE time		(0.1s-0.5s)	(35s-20min)	—	(0.4s-5min32s)
Geo. Mean		292.1 ×	200.9×	64.7×	91.8×

StreamHLS [8], which automatically generates a testbench to validate functional equivalence by comparing the accelerator outputs with the golden results of the original program.

VIII. EXPERIMENTS

Setup. We evaluate performance with Xilinx Vitis HLS and Vivado 2023.2 for synthesis and hardware implementation. Latency and resource statistics are collected from HLS synthesis reports, and runtime and power statistics are measured through on-board evaluation. The platform is an AMD Alveo U280 FPGA board, containing 9024 DSP slices, 2.6M flip-flops, 1.3M LUTs, and 4032 BRAM18K blocks. The target frequency is set to 300 MHz for all experiments.

Baseline and Workload. We compare CODO with six compilers, including ScaleHLS [43], POM [46], Allo [10], HIDA [44], StreamHLS [8], and StreamTensor [42], and an accelerator, DFX [17]. We begin by evaluating performance on typical kernel-level applications from PolyBench and widely used models [16], [18], [38], [39]. We further compare these frameworks on more complex DNN workloads, including ResNet-18 [16], VGG-16 [37], MobileNet [18], ZFNet [45], and YOLO [36]. To demonstrate CODO’s applicability to large language models (LLMs), we also compare CODO against other frameworks on a transformer-based model, GPT-2 [35].

A. Evaluation on Typical Kernel-level Applications

The comparison is shown in Table II, presenting the *latency speedup* along with the resource usage of CODO. The *latency speedup* is computed through dividing the latency (#clock cycles) of the Vitis HLS-optimized code by that of the framework-optimized code. We set the same resource budget (DSP = 900, about 1/3 of the DSPs of a single super logic region) for all the frameworks. Allo, HIDA, and StreamHLS are compared because they focus on dataflow optimization, demonstrating higher performance on applications with multiple kernels compared to ScaleHLS and POM.

For simple kernels with few dataflow optimization opportunities in Polybench, CODO achieves competitive or higher latency speedups. For more complex deep learning workloads, HIDA delivers unsatisfying performance due to lacking support of FIFO-based dataflow, while Allo suffers from performance degradation due to the lack of automated scheduling.

TABLE III: Evaluation on different DNN models with input size 3*32*32.

Application	Framework	Latency (cycles)	Speedup	Compilation Time(s)	BRAM (Util.%)	DSP (Util.%)	FF (Util.%)	LUT (Util.%)
ResNet-18	ScaleHLS	104.88M	5.3×	60.8	8416 (208.7%)	1330 (14.7%)	144K (5.5%)	992K (76.1%)
	POM	20.33M	27.4×	77.3	0 (0.0%)	577 (6.4%)	22K (0.9%)	90K (6.9%)
	Allo	8.29M	66.9×	-	0 (0.0%)	652 (7.4%)	51K (3.5%)	124K (9.8%)
	CODO	1.69M	326.6×	1.2	116 (2.9%)	468 (5.2%)	22K (0.9%)	103K (7.9%)
VGG-16	ScaleHLS	28.31M	6.8×	37.3	3936 (97.6%)	882 (9.8%)	100K (3.8%)	714K (54.8%)
	POM	10.16M	18.9×	57.8	0 (0.0%)	416 (4.6%)	19K (0.7%)	75K (5.7%)
	Allo	3.85M	50.1×	-	0 (0.0%)	440 (4.9%)	36K (1.4%)	98K (7.5%)
	CODO	1.22M	158.0×	1.0	60 (1.7%)	376 (4.2%)	16K (0.6%)	79K (6.1%)
MobileNet	ScaleHLS	2.17M	5.6×	38.1	6796 (168.6%)	1778 (19.7%)	93K (3.6%)	518K (39.7%)
	POM	2.02M	6.0×	139.0	0 (0.0%)	928 (11.7%)	33K (1.7%)	143K (13.1%)
	Allo	0.26M	46.6×	-	0 (0.0%)	1942(21.5%)	57K (2.2%)	128K (9.8%)
	CODO	0.08M	151.5×	1.1	70 (1.7%)	220 (2.4%)	14K (0.6%)	62K (4.8%)

TABLE IV: Evaluation on different DNN models with input size 3*224*224 (except YOLO: 3*1280*384).

Application	Framework	Latency (cycles)	Speedup	Compilation Time(s)	BRAM (Util.%)	DSP (Util.%)	FF (Util.%)	LUT (Util.%)
ResNet-18	HIDA	74.85M	29.7×	83.1	1857 (46.1%)	574 (6.4%)	48K (1.8%)	132K (10.2%)
	CODO	4.76M	466.5×	1.4	548 (13.6%)	535 (5.9%)	26K (1.0%)	115K (8.8%)
VGG-16	HIDA	56.93M	83.0×	199.9	2344 (58.1%)	1163 (12.9%)	67K (2.6%)	209K (16.0%)
	CODO	7.85M	601.8×	4.3	141 (3.5%)	952 (10.5%)	32K (1.2%)	167K (12.8%)
MobileNet	HIDA	23.14M	23.4×	110.8	2060 (51.1%)	782 (8.7%)	49K (1.9%)	141K (10.8%)
	CODO	2.02M	268.5×	1.8	256 (6.3%)	677 (7.5%)	23K (0.9%)	123K (9.4%)
ZFNet	HIDA	15.49M	69.9×	116.2	1223 (30.3%)	644 (7.1%)	31K (1.2%)	83K (6.4%)
	CODO	5.48M	197.7×	6.1	68 (1.7%)	630 (7.0%)	20K (0.8%)	91K (7.0%)
YOLO	HIDA	59.64M	83.9×	188.2	1989 (49.3%)	919 (10.2%)	50K (1.9%)	152K (11.7%)
	CODO	11.90M	420.5×	4.4	147 (3.6%)	1132 (12.5%)	53K (2.0%)	171K (13.1%)

StreamHLS fails to eliminate all violations and generates accelerators with discontinuous dataflow regions, leading to sequential or ping-pong-based execution in applications such as the 3-Layer Block and DWSCConv. In contrast, CODO extensively eliminates fine-grained violations and performs communication buffer optimization, achieving 1.45×, 4.52×, and 3.18× latency speedups on average compared to StreamHLS, Allo, and HIDA, respectively. For DSE time, StreamHLS’s MINLP solver takes exponentially increased search time as the application complexity grows, whereas CODO only takes seconds to find a high-performance design for all applications.

B. Evaluation on DNN Models

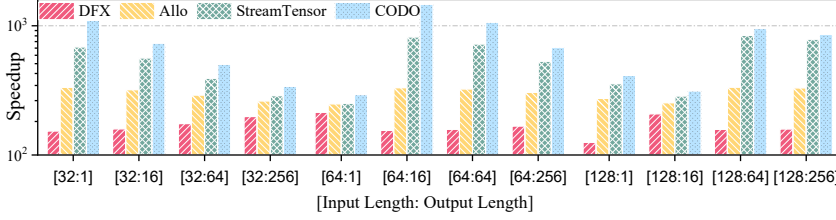
We compare CODO with SOTA frameworks on various DNN models. For fair comparison, we use the same input sizes as in ScaleHLS, POM, and Allo (3*32*32) and extend the comparison with HIDA to ZFNet and YOLO (3*224*224). StreamHLS fails to generate valid designs for large models, and StreamTensor is excluded as it only targets LLM acceleration. The results are shown in Table III and IV.

Performance Comparison. CODO significantly outperforms existing frameworks, achieving average speedups of 33.8×, 13.6×, 3.7×, and 7.1× over ScaleHLS, POM, Allo, and HIDA, respectively, while also reducing compilation time, defined as the time required for all optimizations, DSE, and code generation. The improvement stems from differences in optimization strategy. ScaleHLS and HIDA adopt ping-pong-based dataflow and overlook potential opportunities in more efficient FIFO-based dataflow. POM enhances loop parallelism

and resource reuse, but its strategy quickly reaches a performance ceiling as model size grows. Allo depends on user-specified schedules, often leading to suboptimal pipelines and partitions. In contrast, CODO eliminates both coarse- and fine-grained dataflow violations and prioritizes FIFO implementation, generating high-performance HLS designs.

Resource Usage Comparison. As shown in Tables III and IV, CODO significantly reduces BRAM usage compared to ScaleHLS and HIDA, benefiting from the better resource efficiency of FIFOs over ping-pong buffers. Allo shows 0% BRAM usage because its aggressive parallelism strategy partitions buffers into very small arrays that Vitis HLS maps to LUTRAM, leading to increased LUT consumption. CODO delivers the highest performance with similar or lower resource usage. This efficiency stems from two factors. First, compared to ping-pong-based designs such as ScaleHLS, CODO’s FIFO-based dataflow minimizes on-chip communication latency and stores only in-flight data, thereby achieving higher performance with reduced resource consumption, as discussed in Section II-A. Second, compared to other FIFO-based approaches such as Allo, CODO eliminates more dataflow violations through the co-optimization of correctness, communication, and parallelism, resulting in improved performance. In addition, the parallelism downscaling step removes redundant optimizations on non-critical loops, balancing the dataflow while conserving computation and memory resources.

On-board Verification. We validate CODO’s ability to generate deployable dataflow accelerators on an AMD Alveo U280 FPGA. Among the compared frameworks, ScaleHLS



Framework	Platform	HBM	Freq.	Quant.
DFX	U280	8GB	200MHz	FP16
Allo	U280	8GB	250MHz	W4A8
StreamTensor	U55C	16GB	250MHz	W4A8
CODO	U280	8GB	300MHz	W4A8

Fig. 9: **Left:** Latency speedup over baseline on the GPT-2 model. **Right:** Experiment setup of evaluated platforms.

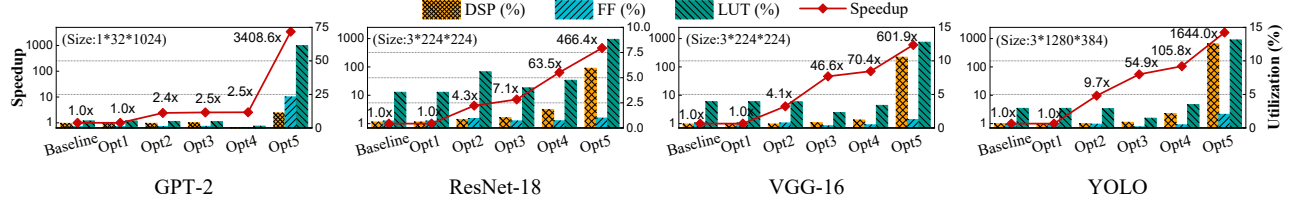


Fig. 10: Ablation study of different optimization methods.

TABLE V: On-board Verification of DNN models.

Application (input size)	Framework	Overall Speedup	Comp. Speedup	P(W)	Exec. Time(s)	E(J)
ResNet-18 (3*32*32)	Baseline	1x	1x	34.7	1.839	63.8
	CODO	69.2x	70.9x	30.7	0.026	0.8
VGG-16 (3*32*32)	Baseline	1x	1x	33.9	0.646	21.9
	CODO	51.0x	55.9x	31.1	0.013	0.4
MobileNet (3*32*32)	Baseline	1x	1x	34.5	0.041	1.4
	CODO	9.6x	9.8x	31.0	0.004	0.1
ResNet-18 (3*224*224)	Baseline	1x	1x	31.8	7.364	234.0
	HIDA	23.9x	29.1x	32.8	0.308	10.1
	CODO	100.6x	101.6x	31.1	0.073	2.5
VGG-16 (3*224*224)	Baseline	1x	1x	34.2	15.825	541.2
	HIDA	14.3x	82.7x	34.2	1.108	37.9
	CODO	127.5x	138.5x	31.2	0.124	3.9
MobileNet (3*224*224)	Baseline	1x	1x	34.7	1.689	58.6
	HIDA	10.0x	21.8x	31.4	0.169	5.3
	CODO	43.8x	44.2x	33.3	0.039	1.3
ZFNet (3*224*224)	Baseline	1x	1x	30.8	3.692	113.7
	HIDA	6.2x	61.5x	32.2	0.593	19.1
	CODO	110.7x	130.9x	29.9	0.034	1.0

fails due to excessive memory usage, Allo encounters deadlocks from fine-grained dataflow violations, and StreamHLS cannot generate valid designs for any workload. CODO and HIDA successfully generate executable accelerators for all models except YOLO, where inaccurate resource estimation in Vitis HLS prevents implementation. Table V shows the on-board evaluation results of DNN models, including runtime speedup, power and energy consumption, and execution time. The overall speedup denotes end-to-end performance, including off-chip data communication, whereas comp. speedup focuses exclusively on computation kernels. CODO's higher performance also enables it to achieve the lowest energy consumption, with 77.1x and 9.2x energy efficiency on average compared to the baseline and HIDA, respectively.

C. Evaluation on the GPT-2 Model

To demonstrate CODO's applicability to LLM workloads, we evaluate performance on GPT-2 Medium [31]. Allo and DFX provide manually optimized implementations of GPT-2, and StreamTensor automatically generates optimized GPT-2 accelerators. The remaining compilers do not support transformer models and are therefore excluded from comparison.

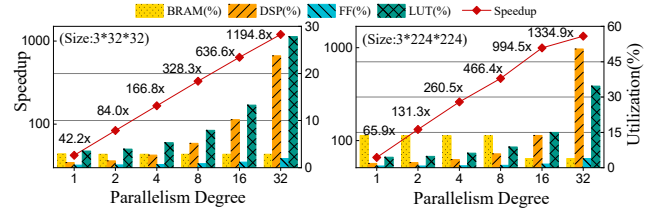


Fig. 11: Latency speedup and resource usage of ResNet-18 under different degrees of parallelism.

The evaluation setup and latency speedups over Vitis HLS-optimized baselines across different input/output sequence lengths are summarized in Fig. 9. Table VI provides a detailed comparison, showing TTFT for prefill performance and Speed for decoding performance.

Compared to manually optimized designs, CODO provides substantial gains. DFX and Allo require hand-crafted implementations and manual coordination of kernels and data transfers, which is time-consuming, error-prone, and often leads to suboptimal dataflow. In contrast, CODO's automatic DSE quickly generates FIFO-based accelerators, resulting in 3.54x and 2.03x speedups over DFX and Allo, respectively.

CODO also outperforms StreamTensor, achieving a 1.23x speedup, even though StreamTensor runs on a more advanced U55C FPGA. Although StreamTensor supports automated DSE, it reverts to a ping-pong execution strategy whenever fine-grained violations are detected, which ultimately limits overall performance. In contrast, CODO eliminates these violations as much as possible and applies communication optimizations to ensure high data transfer efficiency. As a result, CODO surpasses StreamTensor even though StreamTensor executes on an FPGA with larger HBM capacity.

D. Ablation Study

Optimization Method Ablation. To understand the impact of different optimization methods in CODO, we conduct an ablation study across five configurations (Opt1-Opt5), as

TABLE VI: On-board comparison on GPT-2 model. TTFT measures the time to first token, the lower the better. Speed measures the decoding speed in token/s, the higher the better.

[Input Len: Output Len]	DFX			Allo			StreamTensor			CODO		
	Latency (ms)	TTFT (ms)	Speed (token/s)	Latency (ms)	TTFT (ms)	Speed (token/s)	Latency (ms)	TTFT (ms)	Speed (token/s)	Latency (ms)	TTFT (ms)	Speed (token/s)
[32:32]	350.00	177.20	185.19	238.32	81.50	204.05	194.99	34.59	199.51	158.64	20.40	231.48
[64:64]	694.70	349.10	185.19	476.64	162.99	204.05	358.24	61.27	215.51	313.44	32.64	231.48
[128:128]	1384.00	692.80	185.19	953.28	325.98	204.05	696.65	125.35	224.05	663.36	110.40	231.48

TABLE VII: Five configurations of optimization methods.

Optimization	Opt1	Opt2	Opt3	Opt4	Opt5
Coarse-grained Violation Elimination	✗	✓	✓	✓	✓
Fine-grained Violation Elimination	✓	✗	✗	✓	✓
Efficient Data Communication	✗	✗	✓	✓	✓
Automated Dataflow Scheduling	✗	✗	✗	✗	✓

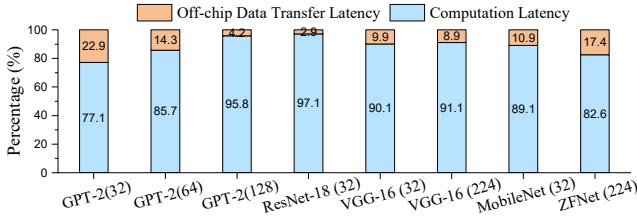


Fig. 12: On-board execution time breakdown.

defined in Table VII. The performance speedups and resource utilization from synthesis results are detailed in Fig. 10. Starting with Opt1, we observe that enabling fine-grained optimizations in isolation yields negligible speedup. This is because unresolved coarse-grained violations invalidate dataflow optimization, leading to sequential execution. In contrast, Opt2 resolves these violations and enables basic ping-pong buffer-based dataflow execution, achieving initial performance gains ranging from $2.5\times$ to $9.7\times$. Next, Opt3 enables efficient data communication. For models with high data reuse potential, such as ResNet-18 and YOLO, the generation of line and window buffers significantly improves on-chip data reuse and communication efficiency, delivering higher speedups compared to Opt2. Building on this, Opt4 addresses fine-grained violations to enable efficient FIFO-based dataflow, boosting performance up to $105.8\times$. Finally, Opt5 delivers the highest performance improvements by leveraging resource-aware parallelism exploration and inter-task optimization. Notably, for computation- and communication-intensive workloads like GPT-2, applying Opt2-Opt4 yields limited gains due to the extremely imbalanced dataflow, which severely hinders overall performance. Opt5 addresses this by enforcing resource-aware parallelism exploration and inter-task optimization, enabling a high percentage of efficient FIFO implementations with a balanced dataflow. This demonstrates that the superior

TABLE VIII: Percentage of FIFO usage.

Application	Gesummv	Residual Block	Multi-Head Attention	MobileNet	ResNet-18	GPT-2
Percentage	100%	100%	84%	100%	100%	89%

performance of CODO results from the joint co-optimization of correctness, communication, and parallelism.

Resource-Performance Trade-off Evaluation. To evaluate CODO’s ability to generate efficient designs under various resource budgets, we conducted a resource-performance trade-off experiment by adjusting the parallelism degree to simulate different resource budgets, as shown in Fig. 11. The results show that performance speedup increases nearly linearly with higher parallelism degrees, accompanied by a steady rise in DSP utilization. This indicates that even on resource-constrained FPGA boards, CODO can generate efficient dataflow accelerators by appropriately tuning parallelism.

On-board Execution Time Breakdown. Figure 12 shows a detailed breakdown of execution time for GPT-2 with different prefill lengths and DNNs with different input sizes. Overall, data transfer time remains low since CODO effectively utilizes HBM bandwidth. For GPT-2, the data transfer portion is relatively high at short prefill lengths but drops quickly as the sequence length grows. This is because computation in self-attention increases much faster than data movement, causing computation latency to dominate at larger prefill lengths. With efficient FIFO implementation and communication optimizations, CODO consistently delivers satisfying performance gains across different input lengths.

FIFO Percentage Quantification. To quantitatively evaluate the effectiveness of our approach, Table VIII reports the proportion of FIFOs used across benchmarks. Except for a few cases in attention and GPT-2, where detected optimization strategy conflicts trigger a fallback to ping-pong buffers, all other tasks achieve a 100% FIFO implementation. This demonstrates the strong scalability and effectiveness of CODO’s dataflow violation elimination.

IX. CONCLUSION

In this paper, we propose CODO, an automated compiler that detects and resolves dataflow violations in the input DNN models, and generates feasible and efficient dataflow accelerators on FPGAs. CODO provides both on- and off-chip optimizations to improve data communication efficiency. A resource-aware automated scheduling method is equipped to generate high-performance dataflow accelerators rapidly.

ACKNOWLEDGEMENTS

This work was supported by the National Natural Science Foundation of China (NSFC) under Grant 62472273 and Grant 62232015, the National Key R&D Program of China under Grant 2022YFB4501400, and the NSFC under Grant U24A20234.

ARTIFACT APPENDIX

The whole compilation stack of CODO will be made available as an open-source project on GitHub soon. Users can download and build the project from the source code. Additionally, a docker image containing a pre-built CODO stack is accessible on Docker Hub, facilitating the reproduction of the experimental results in the paper. To enable fast and convenient compilation and execution, a collection of scripts is provided. Furthermore, a unified script is available for automating the entire workflow of the experiment.

A. Artifact check-list

- **Compilation:** CMake is used to compile the whole project. Our experiments were conducted using CMake version 3.20.3. While versions later than 3.14 are expected to be compatible, we recommend using version 3.20.3 or later for consistency.
- **Run-time environment:** Ubuntu 20.04.6 LTS is compatible for the experiments. Other Linux distributions may also work but have not been tested.
- **Metrics:** Latency, speedup, compilation time, and resource (BRAM, DSP, FF, and LUT) usage.
- **Output:** The experimental results will be displayed in the command line output and will also be saved to the corresponding CSV file.
- **Experiments:** The key results in the paper are reproduced, including results in TABLE II, III, and IV and Fig. 11. A total of 82 experiments are conducted to reproduce these results.
- **How much disk space required (approximately)?:** About 20GB for the Docker image and another 120GB for the Vitis_HLS/Vivado tools.
- **How much time is needed to prepare workflow (approximately)?:** About 10 minutes to download the docker image.
- **How much time is needed to complete experiments (approximately)?:** About 20 hours to complete all synthesis experiments.
- **Publicly available?:** Yes, the source code of CODO will be released on GitHub, Docker Hub, and Zenodo.

B. Description

1) *How to access:* A docker image containing a pre-built CODO stack is accessible on Docker Hub for users to try out CODO quickly. The links to the repositories are listed in Section IX-A in the appendix. The repository is also publicly archived on Zenodo.

2) *Software dependencies:* Xilinx Vitis HLS 2023.2 and Xilinx Vivado 2023.2 are required in the environment. Please refer to the Xilinx Vitis archive page for installation instructions. Note that versions earlier than 2023.2 may yield inconsistent experimental results with results in the paper. The experiments are conducted on a Ubuntu 20.04.6 LTS system. It is highly likely that other Ubuntu versions such as 18.04 and 22.04 are compatible with CODO.

C. Installation

The docker image can be downloaded from the Docker Hub repository by the instructions below:

```
$ docker pull xzz11/codo_ae_image:v1
```

When building a new docker container, the directory of Vitis HLS and Vivado need to be mounted. You can verify your Vitis directory with the following instructions:

```
$ ls $(YOUR_VITIS_DIR)
DocNav Downloads Model_Composer
Vitis Vitis_HLS Vivado xic
```

Then the docker container can be built from the provided docker image:

```
$ docker run -it -v $(YOUR_VITIS_DIR):$
(YOUR_VITIS_DIR) -e LC_ALL=en_US.UTF-8
-e LANG=en_US.UTF-8
xzz11/codo_ae_image:v1 /bin/bash
```

D. Experiment workflow

First, build the compilation environment:

```
$ bash ./compail.sh
```

This step takes approximately 2 minute.

A unified script `./run_ae.sh` is provided to automate the entire workflow. Alternatively, users can run all synthesis experiments under the `experiments` directory and extract results as follows:

Reproducing all synthesis experiments:

```
$ cd experiments/
$ bash ./run_all.sh
$ bash ./merge_results.sh
```

The extracted results are stored in `all_result.csv`.

Alternatively, users can reproduce specific figures or tables individually.

Reproducing Fig. 11:

```
$ cd experiments/fig-11/
$ bash run_hls.sh
$ python3 extract_rpt_metrics.py
```

The execution time is approximately 5 hours. Results are stored in `result.csv`.

Reproducing Table II:

```
$ cd experiments/table-2/
$ bash run_codo.sh
$ bash run_syn.sh
$ python3 batch_extract_rpt_metrics.py
```

The execution time is approximately 2 hours. Results are stored in `result.csv`.

Reproducing Tables III and IV:

```
$ cd experiments/table-3\_and\_table-4/
$ bash run_ae.sh
$ bash run_hls.sh
$ python3 batch_extract_rpt_metrics.py
```

The synthesis time is approximately 512 minutes. Results are stored in `result.csv`.

On-board evaluation results (Table V, Table VI, and Figure 9):

The on-board experiments were not conducted for AE purposes, as generating all bitstreams for the evaluated models requires over two weeks and a properly configured U280 environment. Nevertheless, we provide the host and kernel source code, placement-and-route reports, and prebuilt xclbin files for GPT-2 (corresponding to Table VI and Figure 9).

```
$ cd <corresponding_folder>
$ ./host.exe kernel.hw.xclbin
```

Reproducing Fig. 9 (on-board execution):

```
$ cd <corresponding_folder>
$ ./host.exe kernel.hw.xclbin
```

Reproducing all synthesis experiments:

```
$ cd experiments/
$ bash run_all.sh
$ bash extract_rpt_metrics.py
```

REFERENCES

- [1] “Alveo u280,” 2024. [Online]. Available: <https://www.xilinx.com/content/dam/xilinx/publications/product-briefs/alveo-u280-product-brief.pdf>
- [2] “Intel hls,” 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/programmable/683680/23-2/pro-edition-getting-started-guide.html>
- [3] “Torch-mlir project,” 2024. [Online]. Available: <https://mlir.llvm.org/docs/Dialects/Linalg/>
- [4] “Vitis hls dataflow,” 2024. [Online]. Available: <https://docs.amd.com/t/en-US/ug1399-vitis-hls/pragma-HLS-dataflow>
- [5] D. Abts, J. Ross, J. Sparling, M. Wong-VanHaren, M. Baker, T. Hawkins, A. Bell, J. Thompson, T. Kahsai, G. Kimmell *et al.*, “Think fast: A tensor streaming processor (tsp) for accelerating deep learning workloads,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 145–158.
- [6] R. Andri, L. Cavigelli, D. Rossi, and L. Benini, “Yodann: An architecture for ultralow power binary-weight cnn acceleration,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 1, pp. 48–60, 2017.
- [7] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzyniek, and K. Asanović, “Chisel: Constructing hardware in a scala embedded language,” in *DAC Design Automation Conference 2012*, 2012, pp. 1212–1221.
- [8] S. Basalama and J. Cong, “Stream-hls: Towards automatic dataflow acceleration,” in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA ’25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3706628.3708878>
- [9] H. Chen, J. Zhang, Y. Du, S. Xiang, Z. Yue, N. Zhang, Y. Cai, and Z. Zhang, “Understanding the potential of fpga-based spatial acceleration for large language model inference,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 18, no. 1, Dec. 2024. [Online]. Available: <https://doi.org/10.1145/3656177>
- [10] H. Chen, N. Zhang, S. Xiang, Z. Zeng, M. Dai, and Z. Zhang, “Allo: A programming model for composable accelerator design,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, pp. 593–620, 2024.
- [11] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [12] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, “High-level synthesis for fpgas: From prototyping to deployment,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, 2011.
- [13] D. Durst, M. Feldman, D. Huff, D. Akeley, R. Daly, G. L. Bernstein, M. Patrignani, K. Fatahalian, and P. Hanrahan, “Type-directed scheduling of streaming accelerators,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 408–422. [Online]. Available: <https://doi.org/10.1145/3385412.3385983>
- [14] G. Gobieski, S. Ghosh, M. Heule, T. Mowry, T. Nowatzki, N. Beckmann, and B. Lucia, “Riptide: A programmable, energy-minimal dataflow compiler and architecture,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 546–564.
- [15] L. Guo, Y. Chi, J. Lau, L. Song, X. Tian, M. Khatti, W. Qiao, J. Wang, E. Ustun, Z. Fang *et al.*, “Tapa: a scalable task-parallel dataflow programming framework for modern fpgas with co-optimization of hls and physical design,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 4, pp. 1–31, 2023.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [17] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, “Dfx: A low-latency multi-fpga appliance for accelerating transformer-based text generation,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 616–630.
- [18] A. G. Howard, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- [19] S. Huang, K. Wu, H. Jeong, C. Wang, D. Chen, and W.-M. Hwu, “Pylog: An algorithm-centric python-based fpga programming and synthesis flow,” *IEEE Transactions on Computers*, vol. 70, no. 12, pp. 2015–2028, 2021.
- [20] L. Jia, Z. Luo, L. Lu, and Y. Liang, “Tensorlib: A spatial accelerator generation framework for tensor algebra,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 865–870.
- [21] L. Josipović, R. Ghosal, and P. Jenne, “Dynamically scheduled high-level synthesis,” in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 127–136. [Online]. Available: <https://doi.org/10.1145/3174243.3174264>
- [22] D. Koeplinger, M. Feldman, R. Prabhakar, Y. Zhang, S. Hadjis, R. Fiszal, T. Zhao, L. Nardi, A. Pedram, C. Kozyrakis, and K. Olukotun, “Spatial: a language and compiler for application accelerators,” *SIGPLAN Not.*, vol. 53, no. 4, p. 296–311, Jun. 2018. [Online]. Available: <https://doi.org/10.1145/3296979.3192379>
- [23] H. Kwon, L. Lai, M. Pellauer, T. Krishna, Y.-H. Chen, and V. Chandra, “Heterogeneous dataflow accelerators for multi-dnn workloads,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 71–83.
- [24] Y.-H. Lai, Y. Chi, Y. Hu, J. Wang, C. H. Yu, Y. Zhou, J. Cong, and Z. Zhang, “Heterocl: A multi-paradigm programming infrastructure for software-defined reconfigurable computing,” in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2019, pp. 242–251.
- [25] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, “Mlir: A compiler infrastructure for the end of moore’s law,” *arXiv preprint arXiv:2002.11054*, 2020.
- [26] X. Lian, Z. Liu, Z. Song, J. Dai, W. Zhou, and X. Ji, “High-performance fpga-based cnn accelerator with block-floating-point arithmetic,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 8, pp. 1874–1885, 2019.
- [27] MLIR Contributors, “MLIR: Multi-Level Intermediate Representation,” <https://mlir.llvm.org>, 2026, accessed: 2026-03-06.
- [28] W. S. Moses, L. Chelini, R. Zhao, and O. Zinenko, “Polygeist: Raising c to polyhedral mlir,” in *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT ’21. New York, NY, USA: Association for Computing Machinery, 2021.
- [29] R. Nigam, S. Atapattu, S. Thomas, Z. Li, T. Bauer, Y. Ye, A. Koti, A. Sampson, and Z. Zhang, “Predictable accelerator design with time-sensitive affine types,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 393–407. [Online]. Available: <https://doi.org/10.1145/3385412.3385974>
- [30] T. Nowatzki, V. Gangadhar, N. Ardalani, and K. Sankaralingam, “Stream-dataflow acceleration,” in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 416–429.
- [31] OpenAI Community, “Gpt2-medium,” <https://huggingface.co/openai-community/gpt2-medium>, 2025, [Online; accessed 17-Nov-2025].
- [32] S. Pouget, M. Lo, L.-N. Pouchet, and J. Cong, “Holistic optimization framework for fpga accelerators,” *ACM Trans. Des. Autom. Electron. Syst.*, Sep. 2025. [Online]. Available: <https://doi.org/10.1145/3769307>
- [33] S. Pouget, L.-N. Pouchet, and J. Cong, “A unified framework for automated code transformation and pragma insertion,” in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 187–198. [Online]. Available: <https://doi.org/10.1145/3706628.3708873>
- [34] —, “A unified framework for automated code transformation and pragma insertion,” in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 187–198. [Online]. Available: <https://doi.org/10.1145/3706628.3708873>
- [35] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:160025533>
- [36] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” *CoRR*, vol. abs/1506.02640, 2015. [Online]. Available: <http://arxiv.org/abs/1506.02640>
- [37] K. Simonyan, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.

- [38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [39] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Mach. Learn. Res.*, vol. 11, p. 3371–3408, Dec. 2010.
- [40] S. Xiang, Y.-H. Lai, Y. Zhou, H. Chen, N. Zhang, D. Pal, and Z. Zhang, "Heteroflow: An accelerator programming model with decoupled data placement for software-defined fpgas," in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2022, pp. 78–88.
- [41] A. Xilinx, "Vitis hls 2023.2," <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-hls.html>, 2024.
- [42] H. Ye and D. Chen, "Streamtensor: Make tensors stream in dataflow accelerators for llms," *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:281333142>
- [43] H. Ye, C. Hao, J. Cheng, H. Jeong, J. Huang, S. Neuendorffer, and D. Chen, "Scalehls: A new scalable high-level synthesis framework on multi-level intermediate representation," in *2022 IEEE international symposium on high-performance computer architecture (HPCA)*. IEEE, 2022, pp. 741–755.
- [44] H. Ye, H. Jun, and D. Chen, "Hida: A hierarchical dataflow compiler for high-level synthesis," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2024, pp. 215–230.
- [45] M. Zeiler, "Visualizing and understanding convolutional networks," in *European conference on computer vision/arXiv*, vol. 1311, 2014.
- [46] W. Zhang, J. Zhao, G. Shen, Q. Chen, C. Chen, and M. Guo, "An optimizing framework on mlir for efficient fpga-based accelerator generation," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 75–90.
- [47] X. Zhang, J. Wang, C. Zhu, Y. Lin, J. Xiong, W.-m. Hwu, and D. Chen, "Dnnbuilder: An automated tool for building high-performance dnn hardware accelerators for fpgas," in *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2018, pp. 1–8.
- [48] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He, "Comba: A comprehensive model-based analysis framework for high level synthesis of real applications," in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2017, pp. 430–437.
- [49] R. Zhao, J. Cheng, W. Luk, and G. A. Constantinides, "Polysca: Polyhedral high-level synthesis with compiler transformations," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2022, pp. 235–242. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/FPL57034.2022.00044>